Flutter Routing

1. What is Routing?

You already know how to navigate using Navigator.push() and Navigator.pop(). Routing is simply a better way to organise that navigation when your app grows.

The Problem With Inline Navigation

Every time you navigate to a new screen, you write this:

```
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => DetailScreen()),
);
```

This works fine for 2–3 screens. But in a larger app this code gets scattered everywhere. If you rename DetailScreen, you have to find and update every single call across your entire project.

Named routes solve this. You register all your screens in one place and navigate using a simple string.

2. Named Routes

Setting Up Named Routes

Everything is declared once inside MaterialApp:

```
MaterialApp(
  initialRoute: '/',        // first screen shown when app launches
  routes: {
    '/':        (context) => HomeScreen(),
    '/detail':  (context) => DetailScreen(),
    '/profile': (context) => ProfileScreen(),
  },
);
```

Then navigating from anywhere in your app becomes:

```
Navigator.pushNamed(context, '/detail');
```

No MaterialPageRoute, no widget reference, just a string. Going back is still the same Navigator.pop(context).

Passing Data Between Screens

You often need to send data forward, a user ID, a product name, or anything else. You pass it as arguments:

Sending side

```
Navigator.pushNamed(
  context,
  '/detail',
  arguments: 'Hello from Home!',
);
```

Receiving side

```
class DetailScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    // extract the argument passed from the previous screen
    final String message =
        ModalRoute.of(context)!.settings.arguments as String;

    return Scaffold(
      appBar: AppBar(title: Text('Detail')),
      body: Center(child: Text(message)),
    );
  }
}
```

Note: You can pass any type — a String, an int, or a custom object like a User class.